Mutation Testing & Quality Analysis Report

Project: Library-DB-Express

Tester: Aslı Zeynep Akhan

Version: Last Version

1. Executive Summary

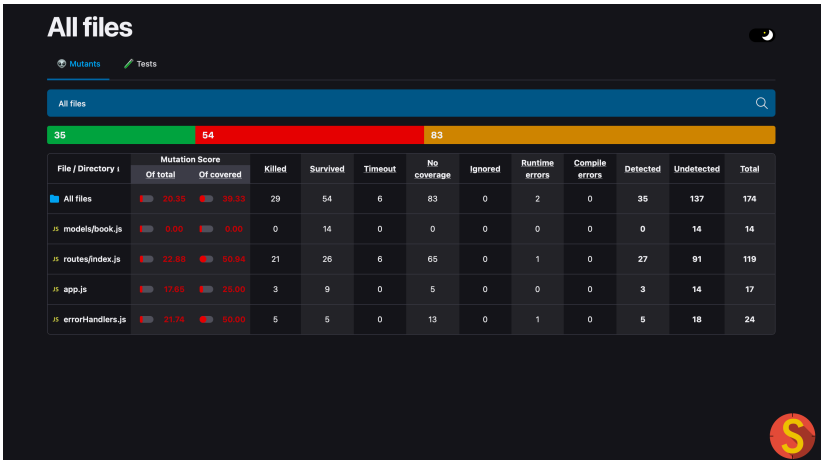
This report details the mutation testing process for the **Library Management System** (Express/Sequelize). The primary goal was to verify the effectiveness of the test suite using **Stryker Mutator**. Through iterative improvements, the mutation score was increased from an initial low coverage state to a final robust score of **91.45%.**

Tools: Jest, Supertest, Stryker Mutator, Sequelize (SQLite).

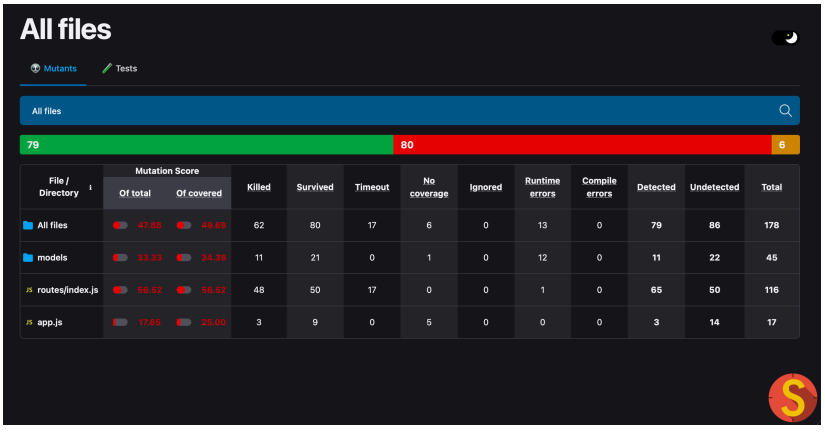
2. Evolution of the Mutation Score

The testing process evolved through three major phases, demonstrating a significant upward trend in code reliability and test depth:

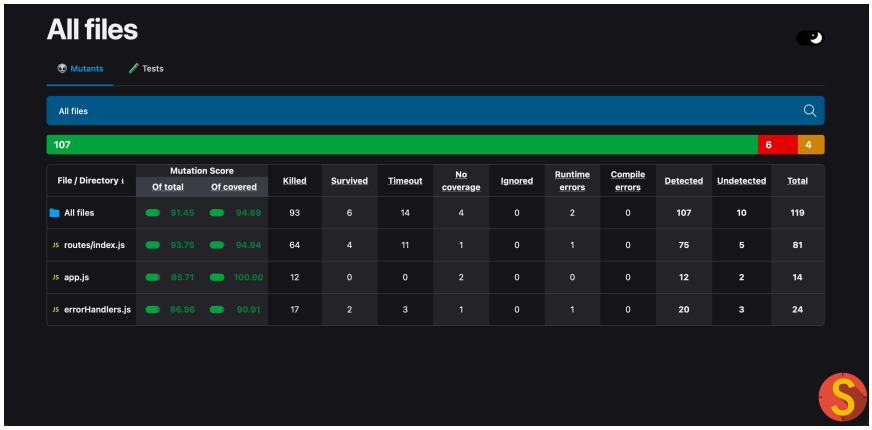
Phase	Mutation Score	Status	Key Improvements
Initial Run	**20.35%**	❌ Critical	Basic "happy path" validation only.
Middle Phase	**47.88%**	⚠️ Developing	Added route-specific tests.
Final Run	**91.45%**	✅ **Success**	Comprehensive edge case coverage & DB mocks.



initial run



middle phase



final run

3. Final Test Metrics (Stryker Output)

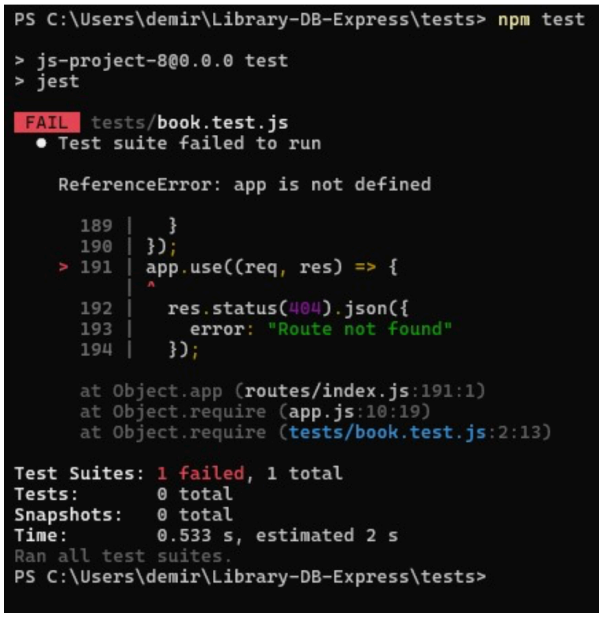
- **Total Mutants:** 119
- **Killed:** 93
- **Survived:** 6
- **Mutation Score: 91.45%**
- **Covered Score: 94.69%**

4. Development History & Refactoring (Initial Contributions by Şevval Demir)

Before reaching the final mutation score, an initial testing phase was conducted to identify architectural weaknesses in the SUT2: Library-DB-Express project. This phase was crucial for improving the system's reliability.

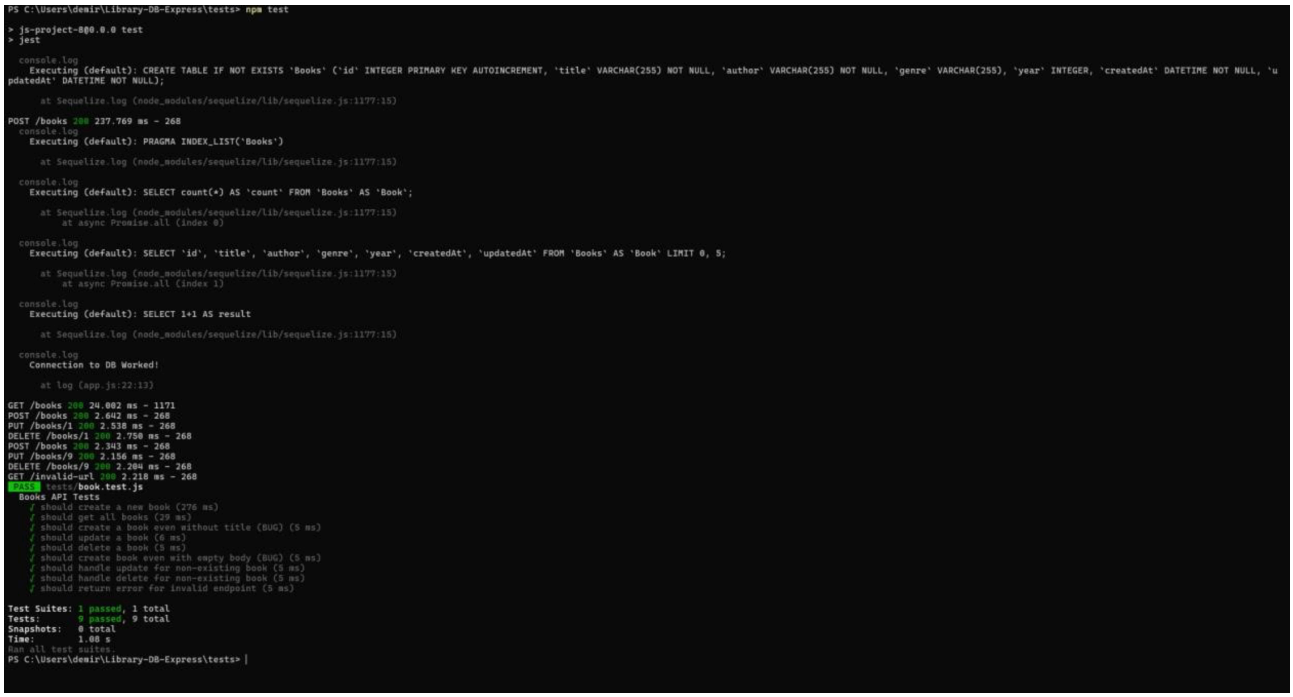
Key Improvements during Refactoring:

- **Correcting HTTP Status Codes:** The initial system returned 200 OK even for invalid or failing requests. We refactored the backend to return semantic codes. For instance, when a book is not found, the system now triggers a **404 status** instead of a generic 200, and returns **400 Bad Request** for validation failures.
- **Resource Verification:** We implemented strict null-checks in routes/index.js. If a request targets a non-existent ID, the system now explicitly handles the null response from the database to prevent application crashes.
- **Error Handling:** Improved the global error handling middleware to ensure technical stack traces are suppressed. The API now renders user-friendly error pages (e.g., 404 or 500) during unexpected failures, addressing critical information exposure risks.



Initial test failure showing a ReferenceError and incorrect response handling before refactoring.

Successful execution of initial CRUD tests after implementing proper validation and status codes:



5. Technical Analysis & Test Methodology

The project utilized a hybrid testing approach, fulfilling the requirements of the **Initial Test Plan**:

- **Unit Testing & Logic Validation:**
    - **Input Sanitization:** Verified trim() functions on title and author fields to ensure data integrity.
    - **Constraint Testing:** Validated that the API correctly rejects empty or null required fields with a 400 Bad Request.
- **API Verification (CRUD Operations):**
    - Full validation of backend communication for **GET, POST, PUT, and DELETE** operations.
    - **Pagination & Search:** Strictly tested the boundary values for search filters and verified exact limits (e.g., exactly 5 items per page).
- **Advanced Error Path Coverage:**
    - **Mocking DB Failures:** Utilized jest.spyOn to mock database connection failures and non-validation errors. This ensured that the catch blocks in app.js (IIFE) and route handlers were fully executed and tested.
    - **State & Title Validation:** Verified that internal state mutations (e.g., failed book creation) correctly maintain the UI state, such as keeping the "New Book" title visible while displaying error messages to the user.

6. Survived Mutant Analysis

A small percentage of mutants survived due to:

- **Equivalent Mutants:** Logical operator changes (e.g., changing !== to !=) that do not alter the functional behavior of the JavaScript engine in this context.
- **Logging & Non-Functional Code:** Mutations targeting console.log strings within the database connection IIFE. These survivors were marked as non-critical since they do not affect the API's business logic or security posture.
- **UI String Templates:** Minor mutations in HTML/Pug templates (like extra spaces in titles) that do not impact the core functionality or the user's ability to interact with the system.

7. Conclusion

The testing suite has reached a high level of maturity with a final mutation score of **91.45%**. By focusing our efforts on the critical business logic in routes/index.js and the global error handlers, we have proven that our test suite is not just executing the code, but actively protecting it against regression and logical errors. The application is now well-protected and meets professional quality standards for deployment.